

Nama: Muhammad Akmal Dwiansyah Putra

Kelas / Absen: TI\_1G / 20

NIM: 254107020110

## **Percobaan Jobsheet 5**

### **Percobaan 1:**

#### **Faktorial20.java:**

```
public class Faktorial20 {  
    int faktorialBF(int n){  
        int fakto = 1;  
        for (int i = 1; i <= n; i++) {  
            fakto = fakto * i;  
        }  
        return fakto;  
    }  
  
    int faktorialDC(int n){  
        if(n == 1){  
            return 1;  
        }else{  
            int fakto = n * faktorialDC(n - 1);  
            return fakto;  
        }  
    }  
}
```

#### **MainFaktorial20.java:**

```
import java.util.Scanner;  
  
public class MainFaktorial20 {  
    public static void main(String[] args) {
```

```

Scanner sc = new Scanner(System.in);

System.out.print("Masukkan Nilai: ");

int nilai = sc.nextInt();

Faktorial20 fk = new Faktorial20();

System.out.println("Nilai Faktorial " + nilai + " menggunakan BF: " + fk.faktorialBF(nilai));

System.out.println("Nilai Faktorial " + nilai + " menggunakan DC: " + fk.faktorialDC(nilai));

sc.close();
}
}

```

#### Hasil Program:

```

Masukkan Nilai: 5
Nilai Faktorial 5 menggunakan BF: 120
Nilai Faktorial 5 menggunakan DC: 120

```

#### 5.2.3. Pertanyaan

1. Pada base line Algoritma Divide Conquer untuk melakukan pencarian nilai faktorial, jelaskan perbedaan bagian kode pada penggunaan if dan else!

**Jawaban:** Base line pada factorial Divide Conquer tersebut adalah jika nilai  $n = 1$  maka akan mereturn nilai 1 juga, sedangkan untuk nilai selain 1 maka akan melakukan perhitungan  $n * n-1$ . Ini dipanggil hingga  $n = 1$  oleh faktorialDC( $n - 1$ )

2. Apakah memungkinkan perulangan pada method faktorialBF() diubah selain menggunakan for? Buktikan!

**Jawaban:** Bisa, berikut perubahan program

```

int i = 1;

while (i <= n) {

    fakto *= i;

    i++;

}

```

#### Hasil:

```

Masukkan Nilai: 5
Nilai Faktorial 5 menggunakan BF: 120
Nilai Faktorial 5 menggunakan DC: 120

```

3. Jelaskan perbedaan antara fakto \*= i; dan int fakto = n \* faktorialDC(n-1); !

**Jawaban:** fakto \*= i hanya bisa digunakan jika ada sebuah loop yang digunakan untuk menghitung semua bilangan, sedangkan fakto n \* faktorialDC(n - 1) digunakan tanpa loop karena loop nya terdapat pada pemanggilan rekursif faktorialDC.

4. Buat Kesimpulan tentang perbedaan cara kerja method faktorialBF() dan faktorialDC()!

**Jawaban:** Dari ini dapat disimpulkan bahwa perhitungan dari kedua fungsi tersebut sama tetapi memiliki tahap berbeda, BF menghitung dari 1 hingga n, sedangkan DC menghitung dari n hingga 1

## Percobaan 2:

### PANGKAT20.JAVA:

```
public class Pangkat20 {  
    int nilai, pangkat;  
  
    Pangkat20(int n, int p){  
        nilai = n;  
        pangkat = p;  
    }  
  
    int pangkatBF(int a, int n){  
        int hasil = 1;  
        for (int i = 0; i < n; i++) {  
            hasil = hasil * a;  
        }  
        return hasil;  
    }  
  
    int pangkatDC(int a, int n){  
        if (n == 1) {  
            return a;  
        }else{  
            if (n % 2 == 1) {  
                return (pangkatDC(a, n/2) * pangkatDC(a, n/2)*a);  
            }else{
```

```

        return (pangkatDC(a, n/2) * pangkatDC(a, n/2));
    }
}
}
}

```

## MAINPANGKAT20.JAVA

```
import java.util.Scanner;
```

```

public class MainPangkat20 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Masukkan jumlah elemen: ");
        int elemen = sc.nextInt();

        Pangkat20[] png = new Pangkat20[elemen];
        for (int i = 0; i < png.length; i++) {
            System.out.print("Masukkan nilai basis elemen ke-" + (i+1) + ": ");
            int basis = sc.nextInt();

            System.out.print("Masukkan nilai pangkat elemen ke-" + (i+1) + ": ");
            int pangkat = sc.nextInt();

            png[i] = new Pangkat20(basis, pangkat);
        }

        System.out.println("HASIL BRUTE FORCE: ");
        for (Pangkat20 p : png) {
            System.out.println(p.nilai+"^"+p.pangkat+": " + p.pangkatBF(p.nilai, p.pangkat));
        }

        System.out.println("HASIL DIVIDE CONQUER: ");
        for (Pangkat20 p : png) {
            System.out.println(p.nilai+"^"+p.pangkat+": " + p.pangkatDC(p.nilai, p.pangkat));
        }
    }
}

```

```

        sc.close();
    }
}

```

## HASIL:

```

Masukkan jumlah elemen: 3
Masukkan nilai basis elemen ke-1: 2
Masukkan nilai pangkat elemen ke-1: 3
Masukkan nilai basis elemen ke-2: 4
Masukkan nilai pangkat elemen ke-2: 5
Masukkan nilai basis elemen ke-3: 6
Masukkan nilai pangkat elemen ke-3: 7
HASIL BRUTE FORCE:
2^3: 8
4^5: 1024
6^7: 279936
HASIL DIVIDE CONQUER:
2^3: 8
4^5: 1024
6^7: 279936

```

### 5.3.3. Pertanyaan

1. Jelaskan mengenai perbedaan 2 method yang dibuat yaitu pangkatBF() dan pangkatDC()!

**Jawaban:** Perbedaan dari pangkatBF() dan pangkatDC() terdapat pada looping perhitungan, BF menggunakan looping for dan i, sedangkan DC menggunakan looping pemanggilan diri sendiri atau rekursif. BF memulai perhitungan dari 1 hingga n sedangkan DC memulai perhitungan dari n hingga 1

2. Apakah tahap combine sudah termasuk dalam kode tersebut? Tunjukkan!

**Jawaban:** Combine pada program tersebut terdapat pada pemanggilan / rekursif dari pangkatDC()

```

if (n % 2 == 1) {
    return (pangkatDC(a, n/2) * pangkatDC(a, n/2)*a);
} else {
    return (pangkatDC(a, n/2) * pangkatDC(a, n/2));
}

```

Menggabungkan hasil yang di dapatkan menjadi 1 bari jawaban

3. Pada method pangkatBF() terdapat parameter untuk melewati nilai yang akan dipangkatkan dan pangkat berapa, padahal di sisi lain di class Pangkat telah ada atribut nilai dan pangkat, apakah menurut Anda method tersebut tetap relevan untuk memiliki parameter? Apakah bisa jika method tersebut dibuat dengan tanpa parameter? Jika bisa, seperti apa method pangkatBF() yang tanpa parameter?

**Jawaban:** bisa, pangkatBF() dapat dijalankan tanpa parameter asalkan isi parameter sebelumnya diganti menjadi nilai dan pangkat, seperti potongan ini

```

int pangkatBF(){

```

```

int hasil = 1;
for (int i = 0; i < pangkat; i++) {
    hasil = hasil * nilai;
}
return hasil;
}

```

4. Tarik tentang cara kerja method pangkatBF() dan pangkatDC()!

**Jawaban:**

BF:

Cara kerja dari pangkatBF() adalah dengan memanggil parameter a dan n, a sebagai angka dan n sebagai pangkat, lalu menggunakan for loop mengkali hasil dengan variable a sebanyak n kali

DC:

Cara kerja dari pangkat DC() adalah dengan memanggil parameter a dan n, sama seperti sebelumnya a sebagai angka dan n sebagai pangkat, pada perhitungan dilakukan pemilihan dimana jika n sama dengan 1 maka akan mereturn 1, selain itu maka akan masuk ke pemilihan bersarang. Pada pemilihan bersarang ini di cek apakah n modulus 2 sama dengan 1, jika iya maka akan memanggil diri nya atau rekursif dikali dengan rekursif dikali a. Jika hasil modulus bukan 1 maka, rekursif dikali rekursif saja. Setiap pemanggilan rekursif ini, n dibagi 2.

**Percobaan 3**

**SUM20.JAVA**

```

public class Sum20 {
    double keuntungan[];

    Sum20(int el){
        keuntungan = new double[el];
    }

    double totalBF(){
        double total = 0;
        for (int i = 0; i < keuntungan.length; i++) {
            total = total + keuntungan[i];
        }
    }
}

```

```

        return total;
    }

    double totalDC(double arr[], int l, int r){
        if(l == r){
            return arr[l];
        }

        int mid = (l+r)/2;
        double lsum = totalDC(arr, l, mid);
        double rsum = totalDC(arr, mid+1, r);
        return lsum+rsum;
    }
}

```

## **MAINSUM20.JAVA**

```

import java.util.Scanner;

public class MainSum20 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Masukkan Jumlah Elemen: ");
        int elemen = sc.nextInt();

        Sum20 sum = new Sum20(elemen);
        for (int i = 0; i < elemen; i++) {
            System.out.print("Masukkan Keuntungan ke-" + (i+1) + ": ");
            sum.keuntungan[i] = sc.nextInt();
        }

        System.out.println("Total keuntungan menggunakan Brute Force: " + sum.totalBF());
        System.out.println("Total keuntungan menggunakan Divide Conquer: " +
            sum.totalDC(sum.keuntungan, 0, elemen-1));
    }
}

```

```

        sc.close();
    }

}

```

**Hasil:**

```

Masukkan Jumlah Elemen: 5
Masukkan Keuntungan ke-1: 10
Masukkan Keuntungan ke-2: 20
Masukkan Keuntungan ke-3: 30
Masukkan Keuntungan ke-4: 40
Masukkan Keuntungan ke-5: 50
Total keuntungan menggunakan Brute Force: 150.0
Total keuntungan menggunakan Divide Conquer: 150.0

```

### 5.4.3. Pertanyaan

1. Kenapa dibutuhkan variable mid pada method TotalDC()?

**Jawaban:** Dibutuhkan variable mid pada totalDC() karena untuk membagi masalah menjadi sub-masalah atau masalah yang lebih kecil, ini agar mudah/cepat untuk dikerjakan sekaligus

2. Untuk apakah statement di bawah ini dilakukan dalam TotalDC()?

```

double lsum = totalDC(arr, l, mid);
double rsum = totalDC(arr, mid+1, r);

```

**Jawaban:** Kegunaan dari kedua potongan program tersebut adalah untuk menggabungkan total dari angka bagian kiri, dan angka bagian kanan masing-masing

3. Kenapa diperlukan penjumlahan hasil lsum dan rsum seperti di bawah ini?

```

return lsum+rsum;

```

**Jawaban:** Kegunaan dari potongan program tersebut adalah untuk menggabungkan total dari kiri dan kanan menjadi 1 dan mereturn hasilnya

4. Apakah base case dari totalDC()?

**Jawaban:** Base case dari totalDC() adalah

```

if(l == r){
    return arr[l];
}

```

Ini menyatakan jika r sama dengan l atau 0, maka mereturn arr[l]



5. Tarik Kesimpulan tentang cara kerja totalDC()

**Jawaban:** Cara kerja dari totalDC() adalah method mengambil parameter yang telah diberikan yaitu array, index pertama, dan index terakhir array. Ini kemudian dibagi menjadi 2 oleh interger mid, dimana kiri dimulai dari index pertama hingga tengah, dan kanan dimulai dari index tengah + 1 hingga index terakhir. Ini dilakukan menggunakan rekursif sebagai loop nya untuk menggabungkan total kiri dan kanan masing masing. Setelah itu program akan mereturn gabungan hasil dari total kiri dan kanan.